

สคริปต์นำเสนอ

ระบบตรวจจับรอยร้าวบนพื้นผิวคอนกรีต (Concrete Crack Detection)

สำหรับ 12 สไลด์ · แบ่ง 3 คน · เวลาโดยรวม ~8-10 นาที

สไลด์ 1 — หน้าปก (Milk)

กลุ่มเราขอแนะนำโปรเจกต์ Concrete Crack Detection หรือระบบตรวจจับรอยร้าวบนพื้นผิวคอนกรีต โดยใช้เทคนิค image processing แบบดั้งเดิมล้วน ๆ ไม่ใช้ deep learning เลย และวัดผลจริงบน dataset ชื่อ CrackForest จำนวน 118 ภาพ ซึ่งเป็นภาพรอยแตกบนพื้นถนน

สไลด์ 2 — ปัญหาและแรงบันดาลใจ (Milk)

จุดเริ่มต้นมาจากเหตุการณ์แผ่นดินไหวที่กระทบประเทศไทย

ทำให้เรื่องความปลอดภัยของอาคารถูกให้ความสำคัญมากขึ้น รอยร้าวบนคอนกรีตเป็นสัญญาณเตือนแรก ๆ ของการเสื่อมสภาพเชิงโครงสร้าง

แต่การตรวจด้วยสายตาคนมีปัญหา 3 อย่าง หนึ่ง—พื้นที่ใหญ่หรือจุดเข้าถึงยากใช้เวลานานและต้นทุนสูง สอง—ผลลัพธ์ขึ้นกับประสบการณ์ของผู้ตรวจแต่ละคน และสาม—เราอยากได้ตัวช่วยคัดกรองเบื้องต้น ให้คอมพิวเตอร์ชี้ว่าจุดไหนควรให้วิศวกรเข้าไปตรวจซ้ำ

สไลด์ 3 — จุดประสงค์และขอบเขต (Milk)

จุดประสงค์คือ พัฒนาระบบที่ตรวจจับรอยร้าวได้อัตโนมัติ และแยก "รอยร้าวจริง" ออกจากพื้นผิวขรุขระ รุพ-run หรือรอยเปื้อนได้

ระบบนี้ทำงานระดับพิกเซล (pixel-level segmentation) คือตัดสินใจทุกพิกเซลว่าเป็นรอยร้าวหรือไม่ ได้ผลออกมาเป็น binary mask ที่ชี้ตำแหน่งและรูปร่างของรอยร้าว ไม่ใช่แค่บอกว่าทั้งภาพมีหรือไม่มีรอยร้าว (image-level) การวัดผลของเราจึงทำระดับพิกเซลให้สอดคล้องกัน

เงื่อนไขสำคัญคือ ใช้ traditional image processing ด้วย OpenCV ล้วน ไม่ใช่ deep learning

ข้อมูลที่ใช้คือ CrackForest 118 ภาพ ขนาด 480×320 ที่มี ground truth ระดับ pixel

ส่วนที่อยู่นอกขอบเขตคือการประเมินความรุนแรง การวัดความกว้างเป็นมิลลิเมตร

และงานวิดีโอแบบเรียลไทม์

สไลด์ 4 — ภาพรวม pipeline 4 ชั้น (Milk)

ระบบเราเป็น pipeline 4 ชั้นตอน จากรูปภาพต้นฉบับไปจนได้ mask ของรอยร้าว

ไล่จากซ้ายไปขวา—ภาพต้นฉบับ, ภาพหลังปรับ contrast, ผลจาก adaptive threshold ที่ยังมี noise, ผลหลัง morphology, ผลสุดท้ายหลังทำ shape filter และ output

สไลด์ 5 — รายละเอียด 4 ชั้นตอน (Patong)

การทำงานแต่ละชั้นมี ดังนี้

ชั้นที่ 1 Pre-processing — แปลงเป็น grayscale, ลด noise ด้วย Gaussian blur แล้วเพิ่ม contrast

เฉพาะจุดด้วย CLAHE ให้รอยร้าวสีเข้มเด่นขึ้น

ชั้นที่ 2 Segmentation — ใช้ adaptive threshold ซึ่งเทียบความสว่างของแต่ละ pixel กับ neighbor ใน window

ชั้นที่ 3 Morphological — ใช้ closing เชื่อมรอยร้าวที่ขาดเป็นช่วง ๆ และ opening ลบจุด noise เล็ก ๆ

ชั้นที่ 4 Shape Analysis — กรองด้วยพื้นที่และ circularity เก็บเฉพาะเส้นที่เรียว ทั้งจุดที่กลม

ค่าพารามิเตอร์ทุกตัวมาจาก grid search

สไลด์ 6 — วิธีวัดผลให้เชื่อถือได้ (Patong)

วิธีวัดผลเราทำ 3 อย่างเพื่อให้ตัวเลขน่าเชื่อถือ

หนึ่ง—แบ่งข้อมูลเป็น train กับ test หาพารามิเตอร์จาก 59 ภาพ แล้ววัดผลบนอีก 59 ภาพที่ไม่เคยเห็น

สอง—เลือกชุดที่ "ดีและ stable" เราแบ่งภาพฝึกเป็น 5 fold แล้วเลือกชุดที่ได้ค่า mean ลบ standard deviation สูงสุด เพื่อกันผลที่ฟลุ่กกับภาพไม่เกี๊ยะภาพ

สาม—เราไม่ใช่ accuracy เพราะ pixel รอยร้าวมีแค่ 2.29% ต่อให้ทายว่า "ไม่มีรอยร้าว" ทุก pixel ก็ได้ accuracy 0.977 แล้ว เราจึงรายงานด้วย IoU และ Dice แทน

สไลด์ 7 — ตัวชี้วัดที่ใช้ (North)

ตัวชี้วัดทั้งหมดมาจากการนับทีละ pixel เทียบ mask ที่ทายกับ ground truth ดูจากไดอะแกรมจะครับ ส่วนที่ทับกันคือ TP ทายถูก, ส่วนที่ทายเกินคือ FP, ส่วนที่พลาดคือ FN

จากนั้นคำนวณ 4 ค่า Precision คือที่ทายว่ารอยร้าวถูกจริงกี่ %, Recall คือรอยร้าวจริงจับได้กี่ %, IoU คือความซ้อนทับของสองพื้นที่ ซึ่งเป็นตัวมาตรฐานของงาน segmentation และ Dice หรือ F1

ที่เป็นค่าเฉลี่ยฮาร์โมนิกของ precision กับ recall เป็นเลขหลักที่เราใช้ดู performance

สไลด์ 8 — ผลการทดลอง (North)

ผลหลักบนภาพจริงทั้ง 118 ภาพ ได้ Dice 0.5024 ขึ้นมาจากค่าเริ่มต้น 0.3201 ส่วน IoU 0.3533, Precision 0.5273 และ Recall 0.5732

ดูจากตารางพัฒนาการ จะเห็นว่าสิ่งที่เปลี่ยนมากที่สุดคือ precision ที่เพิ่มกว่าเท่าตัว จาก 0.25 เป็น 0.53 หมายความว่าระบบเลิกทายพื้นผิวธรรมดาว่าเป็นรอยร้าว ในขณะที่ recall ลดลงเพียงเล็กน้อยเท่านั้น

สไลด์ 9 — เทียบกับวิธีอื่น (North)

เพื่อความไม่ bias เราจูนทุกวิธีด้วยกระบวนการเดียวกันและทำครบทั้ง 4 ชั้น

ผลคือ adaptive threshold ชนะขาด ชนะ Otsu, Canny และ Sobel รวม 102 จาก 118 ภาพ เหตุผลคือ

Canny กับ Sobel เป็น edge detector ที่ตอบสนองต่อการเปลี่ยนความสว่างที่ชัดเจน

ซึ่งบนผิวถนนมีเยอะมากจาก texture ยางมะตอย ขอบคราบ และเงา ส่วน Otsu ใช้ค่า threshold

ค่าเดียวทั้งภาพ จึงเหมาเอาเงาที่มีตมมาเป็นรอยร้าวด้วย แต่ adaptive threshold ถ้ามำถามที่เจาะจงกว่า

คือ "pixel นี้มืดกว่า neighbor ไหม" จึงแม่นกว่า

สไลด์ 10 — รอบการจูน (เปลี่ยนอะไรบ้าง) (ข้าม)

สไลด์ 11 — ตัวอย่างผลลัพธ์ (Patong)

มาดูตัวอย่างผลลัพธ์ แต่ละแถวคือ ภาพต้นฉบับ, ground truth, ผลการ predict และภาพซ้อนทับ

ฝั่งซ้ายคือกรณีที่ทำงานได้ดี รอยร้าวมี contrast ชัดและพื้นหลังสม่ำเสมอ ส่วนฝั่งขวาคือกรณีที่คะแนนต่ำ

แต่ที่น่าสนใจคือบางเคสไม่ใช่ความผิดของระบบ เช่นภาพ 021 ระบบไปเจอรอยแตกของสีเส้นบนถนน

ซึ่งเป็นรอยแตกจริงแล้ว ground truth ไม่ได้ label ไว้ ตัวระบบก็เลย predict ว่าเป็นรอยแตกด้วย

สไลด์ 12 — การต่อยอดและสรุป (Milk)

pipeline ของทำงานอยู่ที่ 7 มิลลิวินาทีต่อภาพ หรือ 143 ภาพต่อวินาที

จึงเหมาะกับการฝังลงฮาร์ดแวร์ขนาดเล็ก

การต่อยอดมี 4 อย่างครับ/ค่ะ หนึ่งในนั้น—ทำเป็น Edge AI Optimize แล้วรันบนบอร์ด ESP32 หรือ FPGA

ร่วมกับ drone สำรวจ สอง—วัดความกว้างเป็นมิลลิเมตรจริงเพื่อประเมินความรุนแรง และสาม—

ทดสอบกับผิวคอนกรีตในอาคาร เพราะ dataset ที่ใช้เป็นผิวถนนซึ่งยากกว่าโจทย์ตั้งต้น

ทั้งหมดนี้คือโปรเจกต์ของกลุ่มเราครับ/ค่ะ ขอขอบคุณ ค่ะ